

Universidad Rafael Landívar
Facultad de Ingeniería
Licenciatura en Ingeniería en Informática y Sistemas
Manejo e Implementación de Archivos Sección: 2
Docente: Ing. Luis Rojas

PRÁCTICA SEMANA 13

“APIs con swagger y postman”

Estudiantes: Marco Fabrizio Donadio Castellanos
Carné: 1076925

Guatemala 01 de octubre de 2026

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

R// Es un conjunto de reglas que permiten que diferentes aplicaciones se comuniquen entre sí, a través de internet y respetando el protocolo HTTP (Hypertext Transfer Protocol).

2. ¿Qué es un endpoint?

R// Es una dirección digital (URL), o un dispositivo físico, que permite la comunicación entre sistemas. El que sea digital o física depende de la industria.

3. ¿Qué diferencia existe entre una URL y un endpoint?

R// Una URL es una dirección general que sirve para localizar cualquier recurso en internet, mientras que un endpoint es una URL específica con un método HTTP diseñada para que una API reciba y responda solicitudes.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

GET: recupera datos de un servidor. Por lo general, se utiliza para acceder a todos los recursos de un tipo determinado o para acceder a un recurso específico.

POST: crea nuevos recursos. Si se desea añadir una factura a una base de datos, por ejemplo, enviaríamos una solicitud POST. La solicitud POST suele incluir un cuerpo, en donde se especifica los atributos del recurso que se va a crear.

PUT: se utiliza para reemplazar un recurso existente por una versión actualizada. Este método reemplaza el recurso completo con los datos incluidos en el cuerpo de la solicitud.

PATCH: lo utilizamos para actualizar un recurso existente. Similar a PUT, con la diferencia de que PATCH sí permite a los clientes actualizar propiedades específicas de un recurso sin sobrescribir las demás.

DELETE: elimina datos de una base de datos. Cuando se envía una solicitud DELETE, solicita que se elimine el recurso en la URL especificada.

5. ¿Qué es JSON y por qué es común en las APIs?

R// JSON (*JavaScript Object Notation*) es un formato de texto ligero diseñado para almacenar y transportar datos estructurados. Es común en las APIs ya que es un “idioma universal”, mismo que permite que diferentes aplicaciones y servidores se comuniquen sin importar qué lenguaje de programación usen por dentro.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

R// Los códigos de estado HTTP son números de tres dígitos que el servidor web envía como respuesta a una petición realizada por el navegador o cliente. Indican si la solicitud tuvo éxito, si requiere pasos adicionales o si ocurrió un error.

200 OK: cuando la petición se realiza con éxito y sin errores.

404 NOT FOUND: no se encontró el recurso o la página solicitada.

500 INTERNAL SERVER ERROR: hubo un error general e inesperado en el servidor.

7. ¿Qué información se puede encontrar en la respuesta de una API?

R// La respuesta de una API contiene el resultado del procesamiento del servidor, incluyendo un código de estado, encabezados y los datos solicitados.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

R// La principal ventaja de este enfoque es la validación temprana del diseño sin escribir código, lo que

optimiza el ciclo de desarrollo. Esto permite detectar errores de lógica antes de programar y facilita que los equipos trabajen en paralelo.

9. ¿Qué es y para qué sirve Swagger UI?

R// Es una herramienta que genera de forma automática documentación visual e interactiva para una API a partir de su archivo de especificación. Sirve para que cualquier desarrollador pueda entender, consultar y probar los endpoints directamente desde el navegador sin escribir código.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

R// Postman cumple la función de ser una plataforma integral para diseñar, enviar peticiones HTTP, automatizar pruebas y simular respuestas de una API. Sirve para validar que el backend funcione exactamente como se espera en diferentes entornos de desarrollo.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

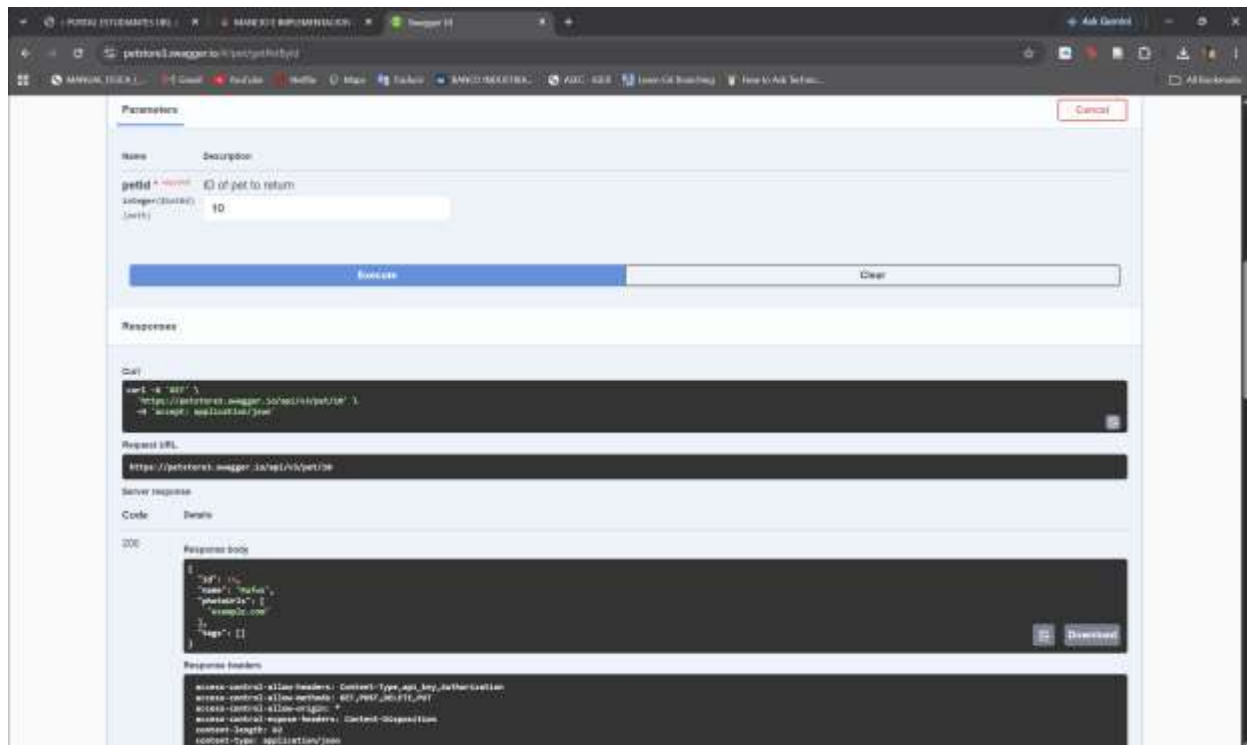
Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
 - Utilizamos el método GET
- Indique qué parámetro necesita la solicitud.
 - Necesita un **petId**. Específicamente, un entero de 64 bits
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
 - Código Obtenido: **200**
- Explique con sus palabras qué información contiene la respuesta.
 - El **“response body”** contiene: el id registrado de la mascota, el nombre de la mascota, una foto de la mascota y sus **“tags”** que, aunque aparecen vacías, imagino que son descripciones o características de la mascota.

Evidencia: captura de Swagger mostrando la operación y su respuesta.



3. Crear un recurso con POST

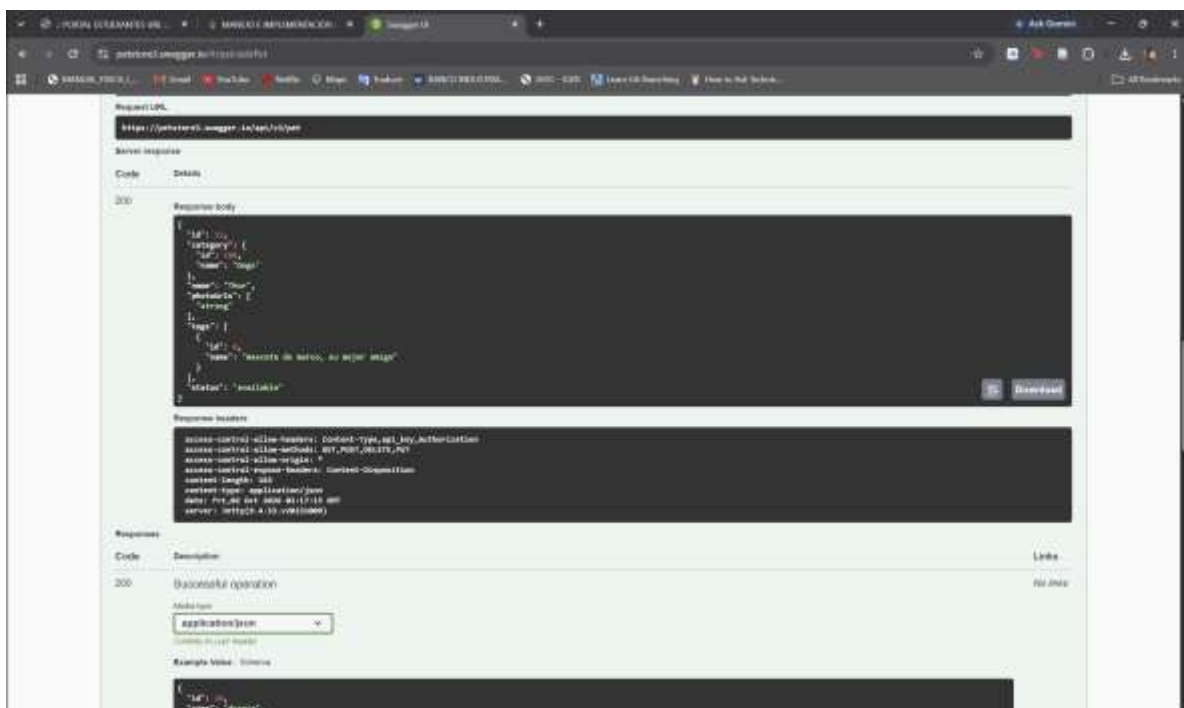
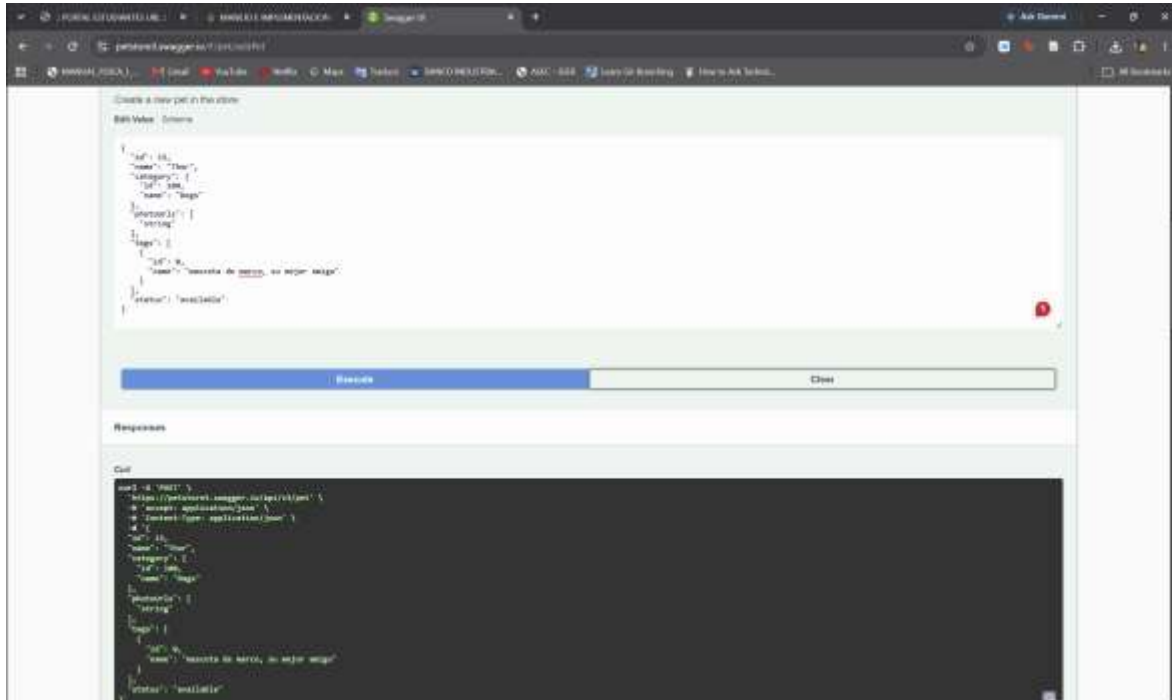
Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

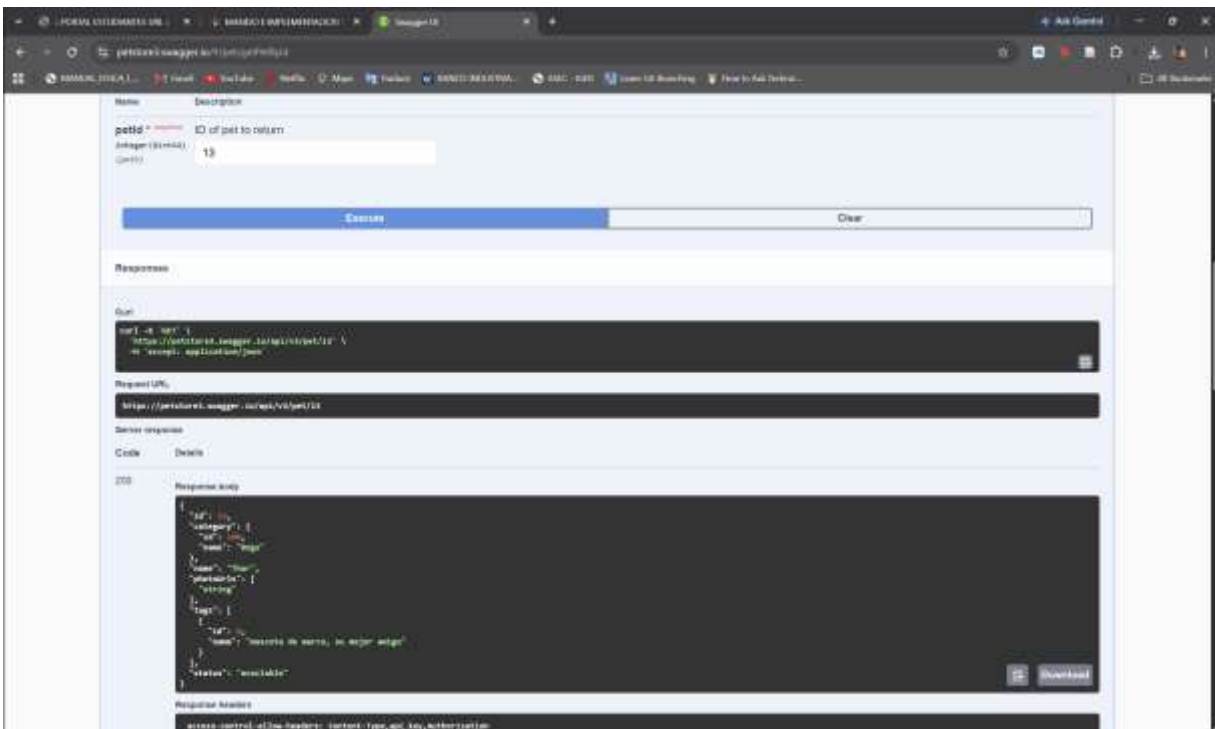
El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

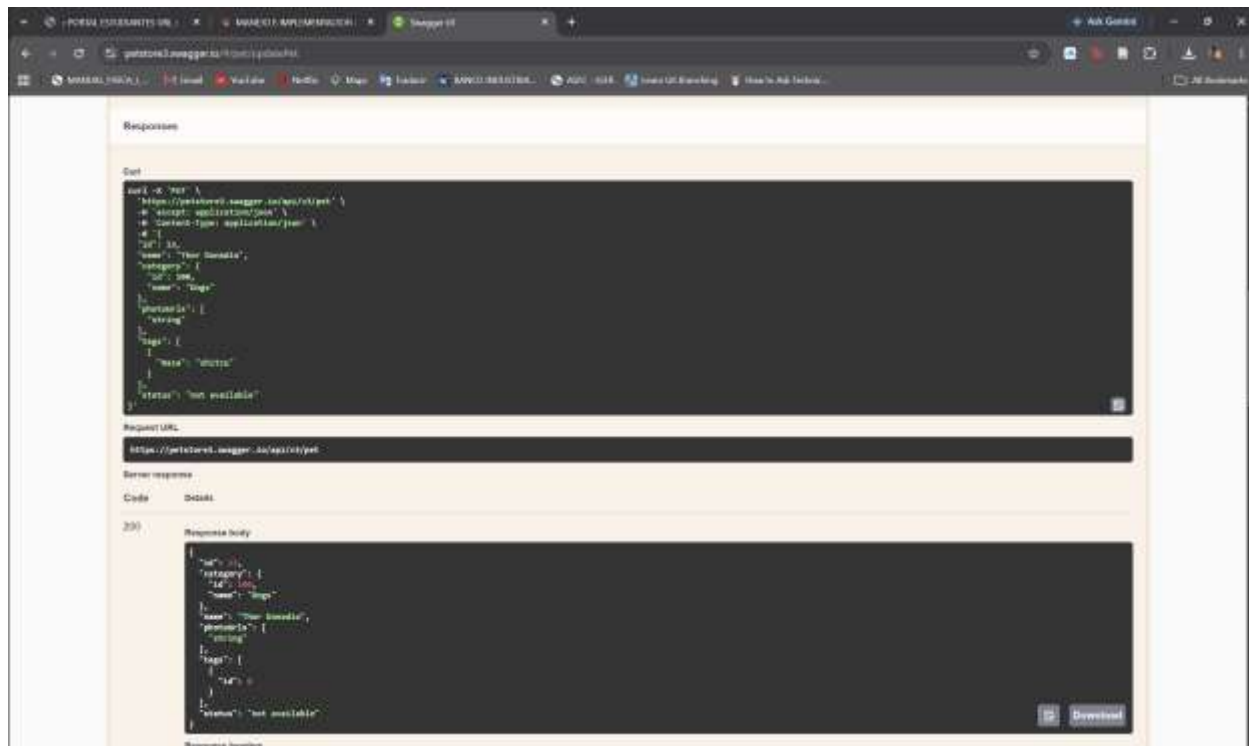




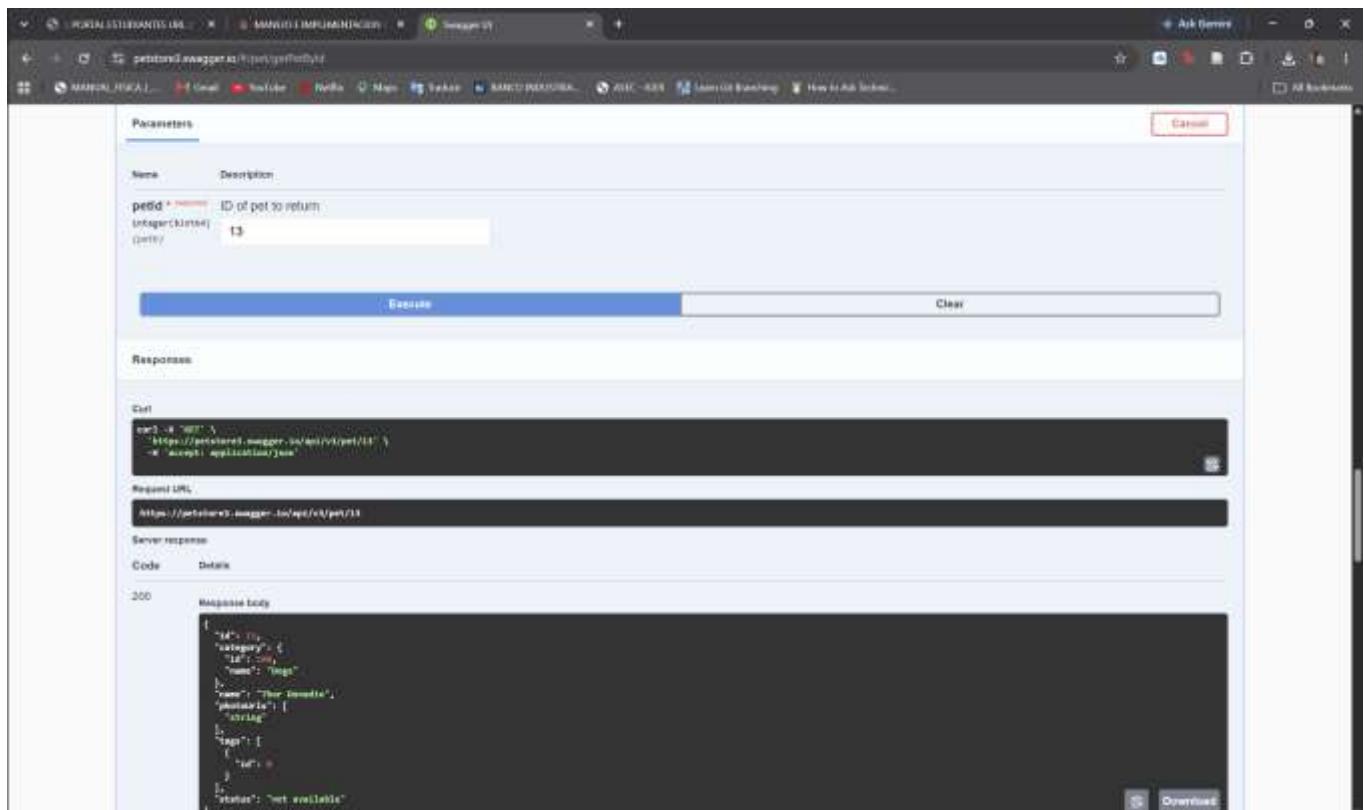
4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
 - Utilicé el método PUT, ya que este funciona especialmente para modificar y actualizar por completo un recurso; en efecto, lo reemplaza por completo con el cuerpo que se le envía.
- ¿Qué datos fueron enviados en el body?


```
{
  "id": 13,
  "name": "Thor Donadio",
  "category": {
    "id": 100,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "Raza": "shitzu"
    }
  ],
  "status": "not available"
}
```
- ¿Qué código HTTP devolvió la API?
 - Devolvió el código: 200



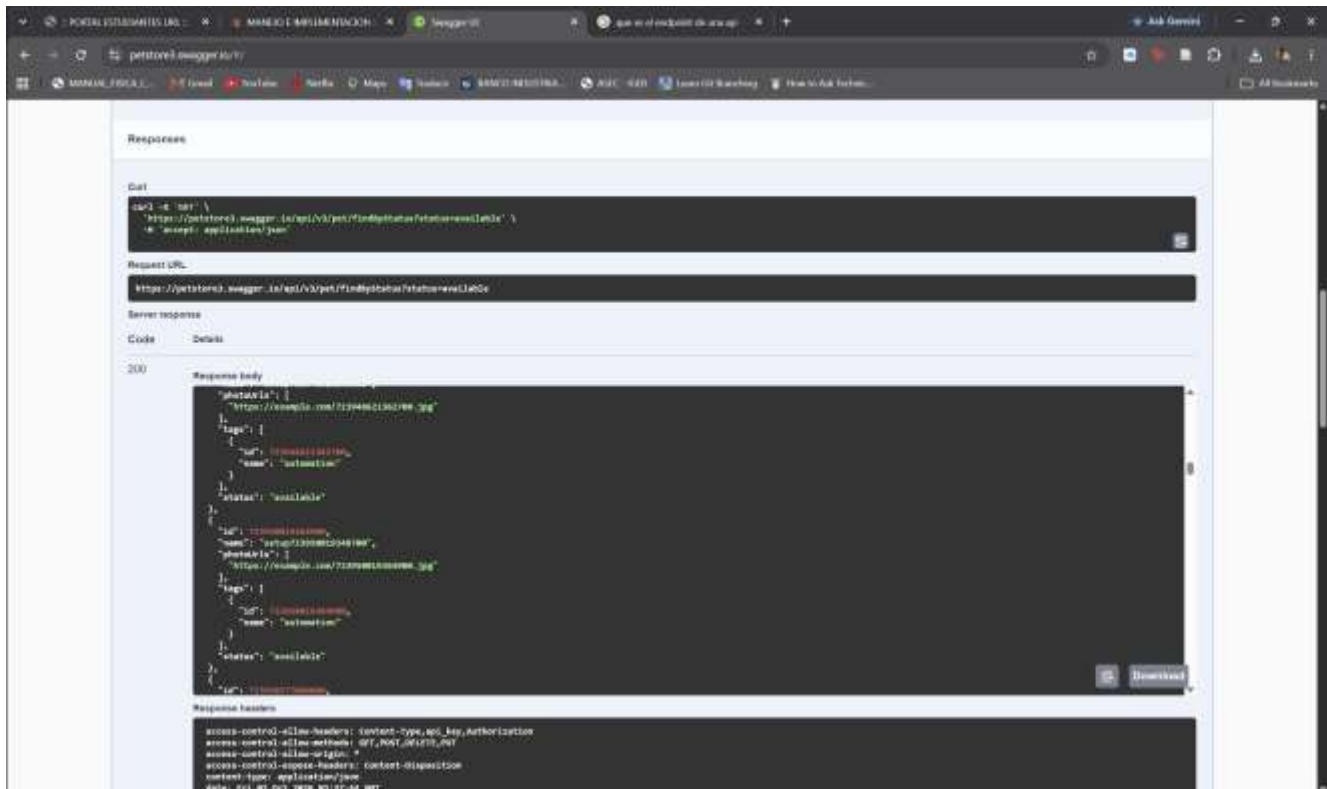
- ¿Cómo comprobó que el cambio realmente ocurrió?
 - Lo verifiqué buscando el id con el método GET y viendo que los cambios realizados estén presentes.



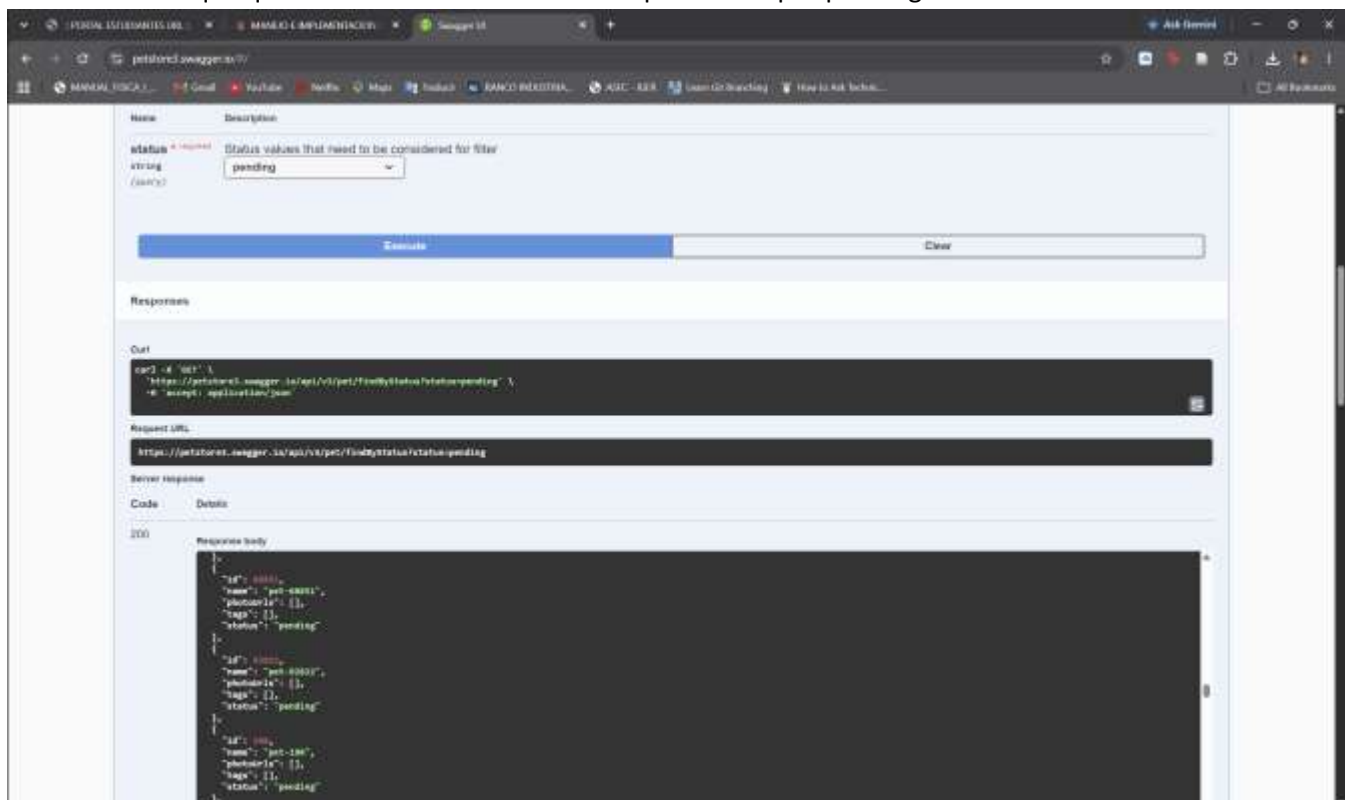
5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.

- Identifique el parámetro utilizado.
 - Parámetro utilizado: **available**
- Explique dónde se envía ese parámetro.
 - Se envía en el **endpoint** (request body):
<https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available>
- Observe la estructura de la respuesta.

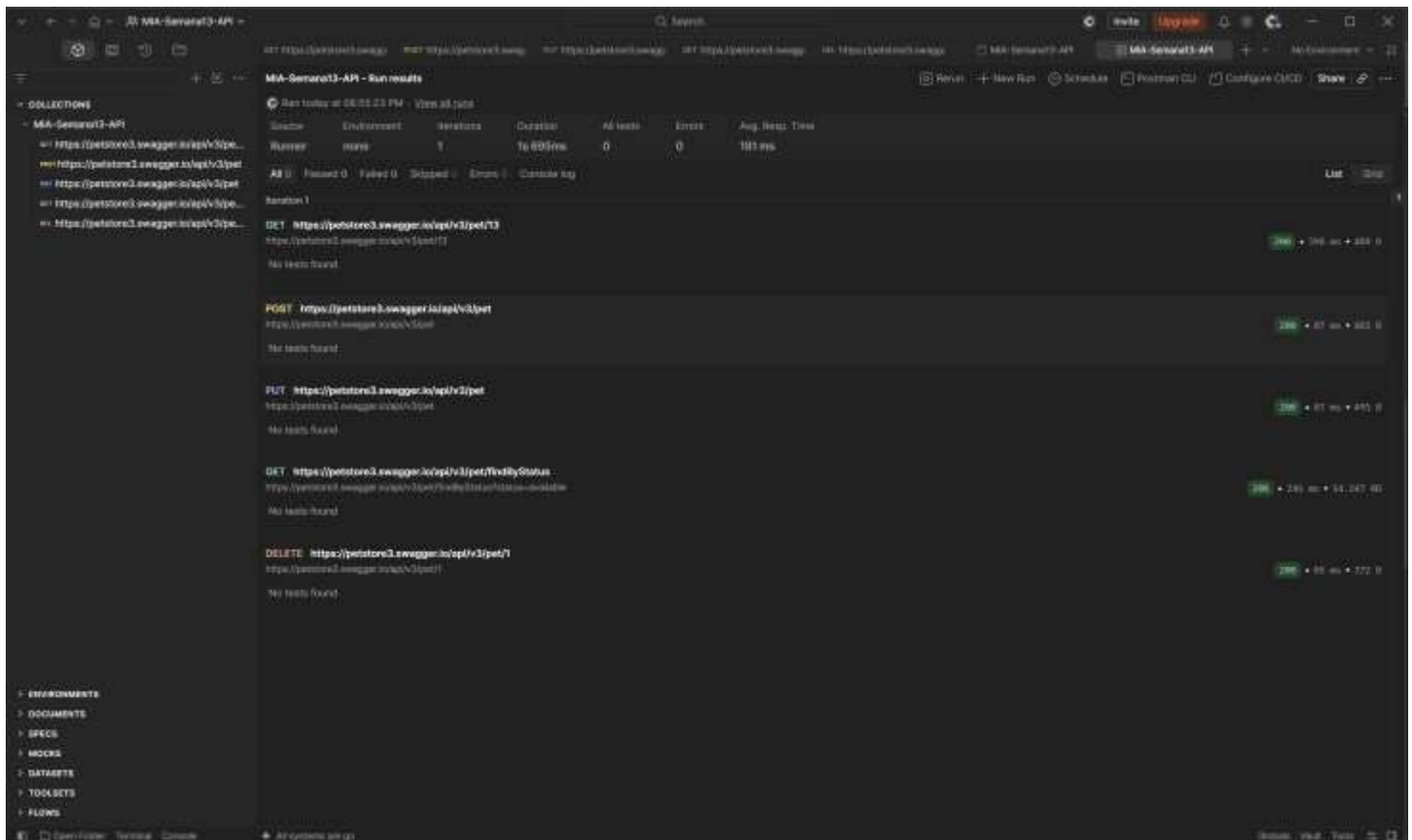


- Indique qué sucede si cambia el valor del parámetro por pending.



6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



Subí la colección de JSON a el repo :D.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

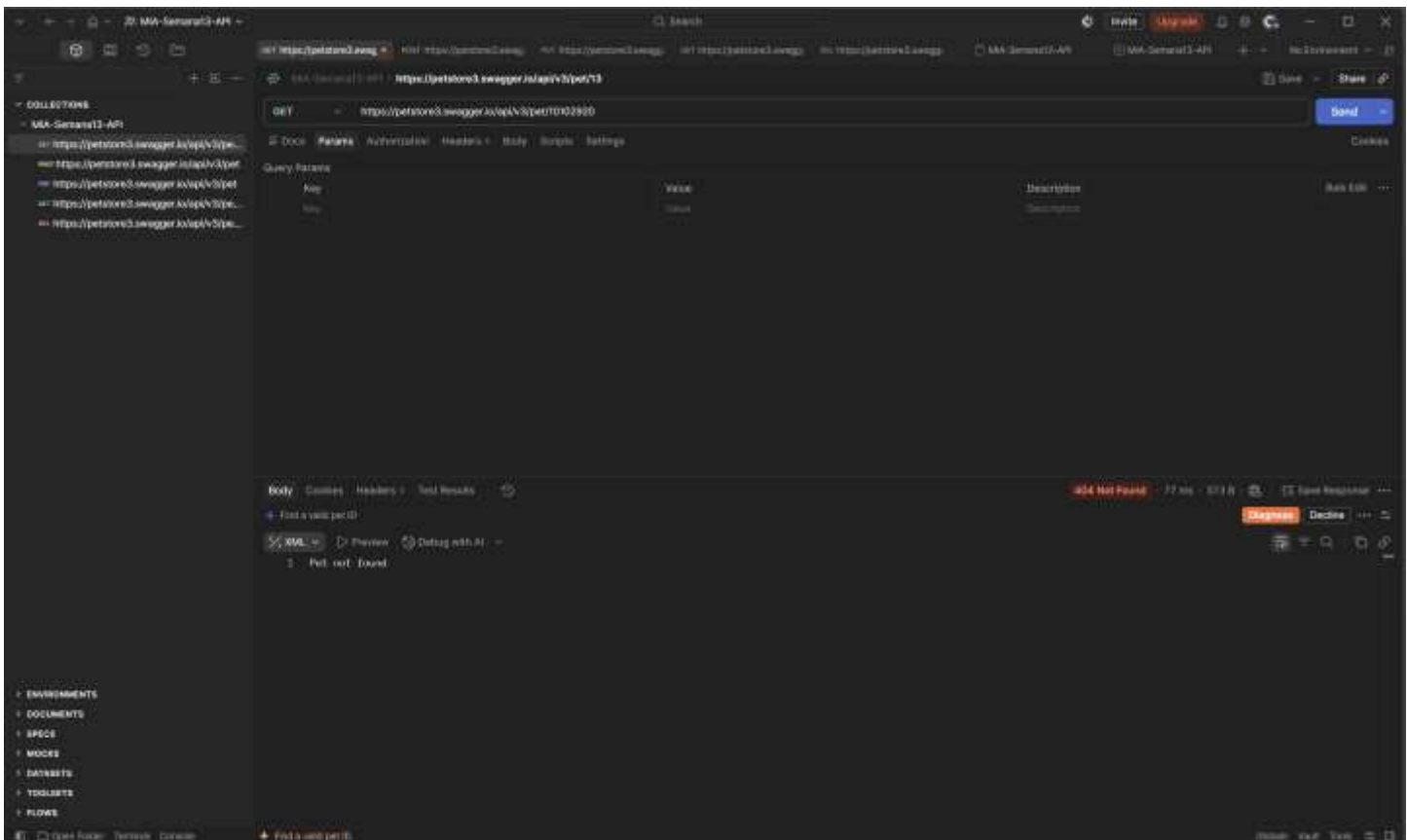
Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

CONSULTA DE ID INEXISTENTE



a) Solicitud realizada:

GET /pet/10102920

b) Código HTTP recibido:

404 Not Found

c) Respuesta de la API:

Pet not found

d) Qué esperaba que ocurriera:

Que la API devolviera los datos de la mascota con ese ID.

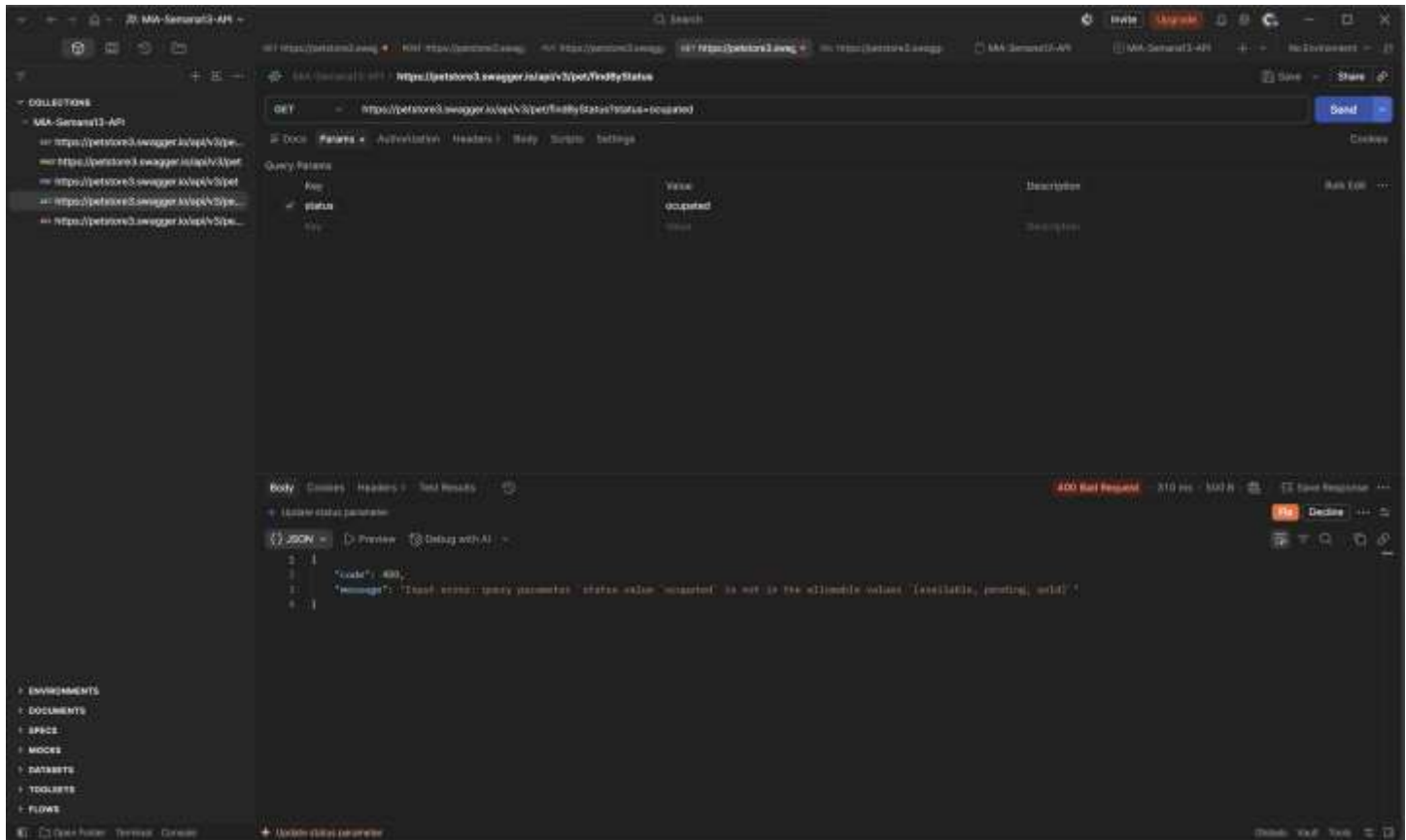
e) Qué ocurrió realmente:

La mascota no fue encontrada.

f) Explicación de la causa:

El ID 10102920 no corresponde a una mascota existente en la API.

UN PARÁMETRO INCORRECTO



a) Solicitud realizada:

GET /pet/findByStatus?status=ocupated

b) Código HTTP recibido:

400 Bad Request

c) Respuesta de la API:

La API indicó que "ocupated" no es un valor permitido.

d) Qué esperaba que ocurriera:

Obtener las mascotas con el estado indicado.

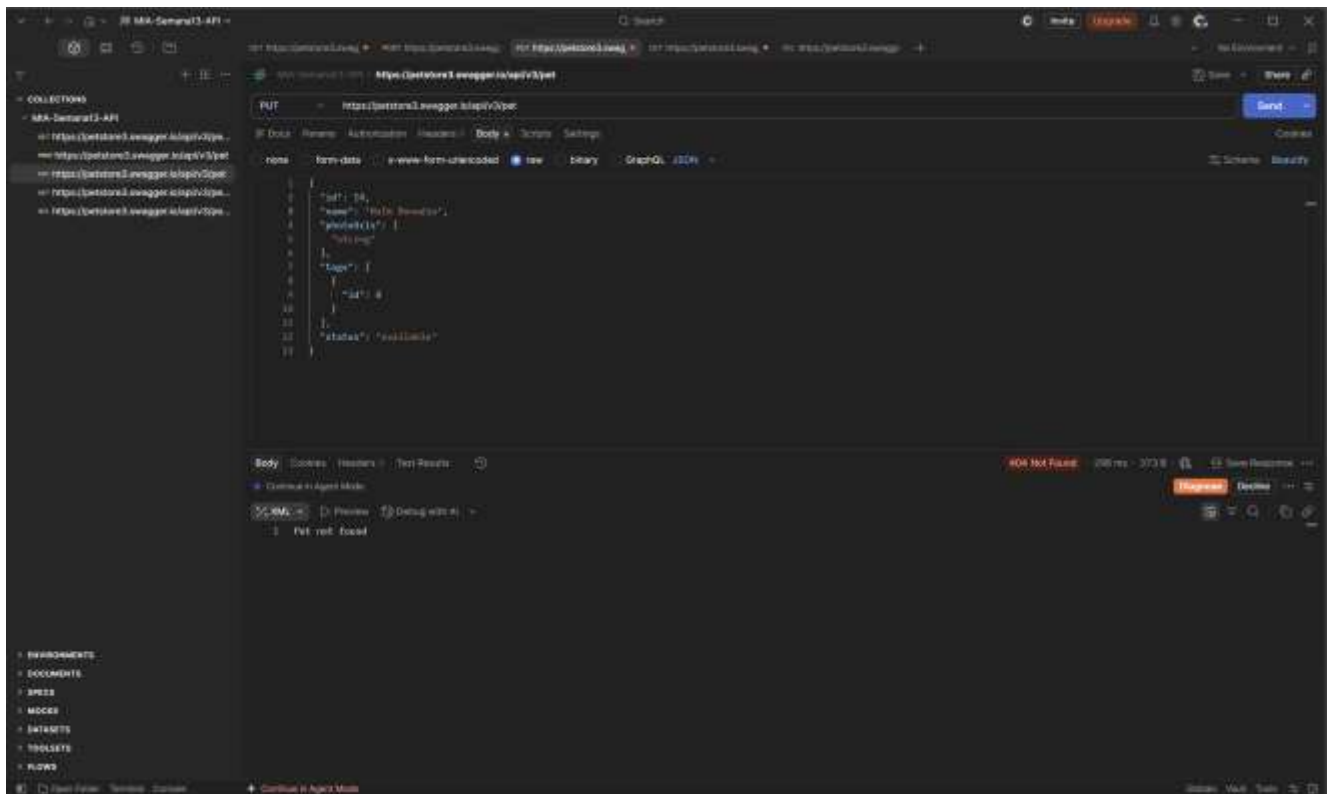
e) Qué ocurrió realmente:

La solicitud fue rechazada por contener un parámetro inválido.

f) Explicación de la causa:

Los estados permitidos son available, pending y sold. El valor ocupated no existe dentro de las opciones válidas.

JSON INCOMPLETO



- a) Solicitud realizada:
PUT /pet enviando una mascota con ID 14 y estado available.
- b) Código HTTP recibido:
404 Not Found
- c) Respuesta de la API:
Pet not found
- d) Qué esperaba que ocurriera:
Que los datos de la mascota fueran actualizados correctamente.
- e) Qué ocurrió realmente:
La API no realizó la actualización porque no encontró la mascota.
- f) Explicación de la causa:
El ID 14 no correspondía a una mascota existente, por lo que no había un registro que pudiera actualizarse.

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Endpoints y métodos HTTP

Operación	Método HTTP	Endpoint propuesto	Respuesta esperada
Registrar producto	POST	/productos	201 Created
Consultar producto	GET	/productos/{id}	200 OK o 404 Not Found
Actualizar producto	PUT	/productos/{id}	200 OK o 404 Not Found
Eliminar producto	DELETE	/productos/{id}	204 No Content o 404 Not Found
Verificar eliminación	GET	/productos/{id}	404 Not Found

Flujo completo

Paso 1 -> Registrar un producto

Se utiliza el método POST para registrar un nuevo producto en el inventario.

Método HTTP: POST

Endpoint: /productos

Cuerpo de la solicitud:

```
{
"nombre": "Coca-Cola 600 ml",
"precio": 8.50,
"stock": 24,
"estado": "Disponible"
}
```

Respuesta esperada: 201 Created

Ejemplo de respuesta:

```
{
"id": 101,
"nombre": "Coca-Cola 600 ml",
"precio": 8.50,
"stock": 24,
"estado": "Disponible"
}
```

Análisis: El código 201 Created confirma que el producto fue registrado correctamente dentro del inventario.

Paso 2 -> Consultar la información del producto

Después de registrarlo, se utiliza el método GET para consultar la información del producto mediante su identificador.

Método HTTP: GET

Endpoint: /productos/101

Respuesta esperada: 200 OK

Ejemplo de respuesta:

```
{
"id": 101,
"nombre": "Coca-Cola 600 ml",
"precio": 8.50,
"stock": 24,
"estado": "Disponible"
}
```

Análisis: El código 200 OK indica que el producto existe y que su información pudo ser consultada correctamente.

Si el producto no existiera, la API debería responder: 404 Not Found

Paso 3 -> Actualizar la disponibilidad del producto

Se utiliza el método PUT para actualizar la información del producto. En este caso, se cambia su estado de Disponible a Agotado.

Método HTTP: PUT

Endpoint: /productos/101

Cuerpo de la solicitud:

```
{  
  "nombre": "Coca-Cola 600 ml",  
  "precio": 8.50,  
  "stock": 0,  
  "estado": "Agotado"  
}
```

Respuesta esperada: 200 OK

Ejemplo de respuesta:

```
{  
  "id": 101,  
  "nombre": "Coca-Cola 600 ml",  
  "precio": 8.50,  
  "stock": 0,  
  "estado": "Agotado"  
}
```

Análisis: El código 200 OK indica que el producto fue actualizado correctamente.

Si el identificador no existe, la respuesta debería ser: 404 Not Found

Paso 4 -> Eliminar el producto

Para eliminar el producto del inventario se utiliza el método DELETE.

Método HTTP: DELETE

Endpoint: /productos/101

Respuesta esperada: 204 No Content

Análisis: El código 204 No Content indica que el producto fue eliminado correctamente y que no es necesario devolver contenido adicional.

Si el producto no existe: 404 Not Found

Paso 5 Verificar que el producto ya no existe

Finalmente, se vuelve a realizar una consulta con GET utilizando el mismo identificador.

Método HTTP: GET

Endpoint: /productos/101

Respuesta esperada: 404 Not Found

Ejemplo de respuesta:

```
{  
  "mensaje": "Producto no encontrado"  
}
```

Análisis: La respuesta 404 Not Found permite confirmar que el producto ya no existe dentro del sistema.

Resumen del flujo

POST → GET → PUT → DELETE → GET

POST: registrar un nuevo producto.

GET: consultar la información del producto.

PUT: actualizar sus datos o disponibilidad.

DELETE: eliminar el producto.

GET final: comprobar que ya no existe.

El flujo propuesto permite administrar los productos de una tienda de conveniencia mediante operaciones HTTP. El método POST se utiliza para registrar productos, GET para consultarlos, PUT para actualizarlos y DELETE para eliminarlos. A su vez, el análisis de los códigos de estado permite conocer el resultado de cada operación. En particular, 404 Not Found permite identificar cuando un producto no existe dentro del sistema